TESTNG

i. Introduction
ii. Basic Annotation
iii. Assertion
iv. Helper attributes
    - @DataProvider
v. Types of execution
    - @Parameters
vi. Advance Report
vii. Configuration Annotations
    - Base class
viii. @Listeners

i. Introduction

1. TestNG is an open source unit testing tool
2. Here, NG stands for Next Generation
3. TestNG comes under TDD framework.
4. Unit testing via

TestNG	Java, .NET
JUnit	Java
NUnit	.NET
PyDev	Python
RSpec	Ruby
Jasmine	JavaScript

5. TestNG supports java and .net.
6. TestNG was developed as an additional plugin for Eclipse
7. In case of automation, TestNG will be used to develop all the scripts using TestNG annotations.
8. TestNG will be used to handle all framework component.
9. TestNG.xml is main controller of the selenium framework, where we start the execution.
10. TestNG is used by automation engineers because of its features and advantages

Installation of TestNG Plugin

1. Open eclipse and click on "help" major tab
2. Select "Eclipse Marketplace".
3. Search "TestNG" in eclipse marketplace dialogue box
4. Click on "install" button under "TestNG for Eclipse"
5. Click on "confirm"
6. Click on "I agree to license agreement" radio button and then "next"
7. Select "install anyway" and then finish
8. Click on select all, trust selected (this box will come twice)
9. Click on "restart now"

Why TestNG over JUnit?

TestNG is a more advanced and flexible testing framework compared to JUnit. It provides additional features that make automation testing more powerful and manageable, especially for large-scale projects.

---

1. **More Annotations**
   - TestNG provides a richer set of annotations compared to JUnit, allowing greater control over test execution flow.
     (e.g., @BeforeSuite, @AfterSuite, @DataProvider, @Factory, etc.)
2. **Batch Execution & Parallel Execution**
   - Multiple test classes or suites can be executed together.
   - Supports parallel execution to reduce total test execution time.
3. **Assertions**
   - Built-in assertion mechanisms for verifying expected results effectively.
4. **Enhanced Functionality**
   - **HTML Reports:** Automatically generates detailed HTML reports after execution.
   - **Parallel Execution:** Runs multiple tests simultaneously for faster feedback.
   - **Grouping Execution:** Tests can be grouped logically (e.g., Smoke, Regression, Sanity).
   - **Additional Annotations:** More control over test configuration and dependency handling.
   - **Simplified Batch Execution:** Easy to configure test batches through the `testng.xml` file.
   - **ITestListeners:** Used for event tracking and custom actions such as capturing screenshots on test failure.
   - **Retry Analyzer:** Automatically re-runs failed test scripts to handle flaky tests.

Advantage:

1. Generate Reports
2. Batch Execution of Test Cases
3. Group Execution of Test Cases
4. Parallel execution of Test Cases
5. Perform Assertion
6. Prioritize the test cases
7. Add dependency of test cases
8. Can disable the test cases.
9. Add preconditions and post condition
10. Provides different annotations

Rules to follow while using TestNG

1. Use @Test method instead of main method
2. Use Reporter.log() instead of printing statement

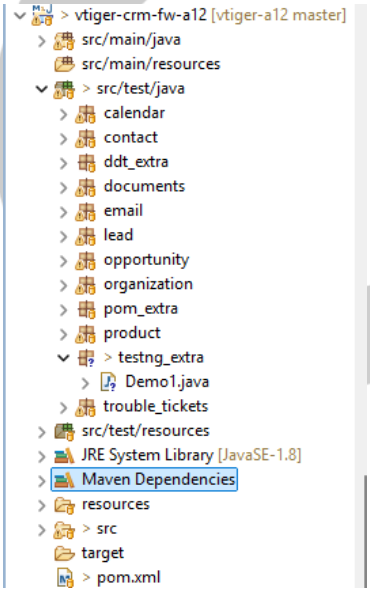
Reporting – Basic reports

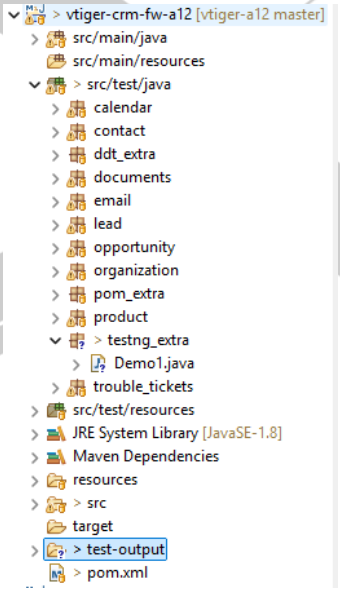
- Basic reports are available in testNG to log the reports into HTML reports
- Reporter class is being used to generate the logs reports
- Test-output -> emailable-report.html (Open with web browser)
- Whenever we want all the details like how and why the test script got failed, which line is having which exception then we will go for Reporter class.
- Replace System.out.println() with Reporter.log();

To Present in console as well as Remains in Reports

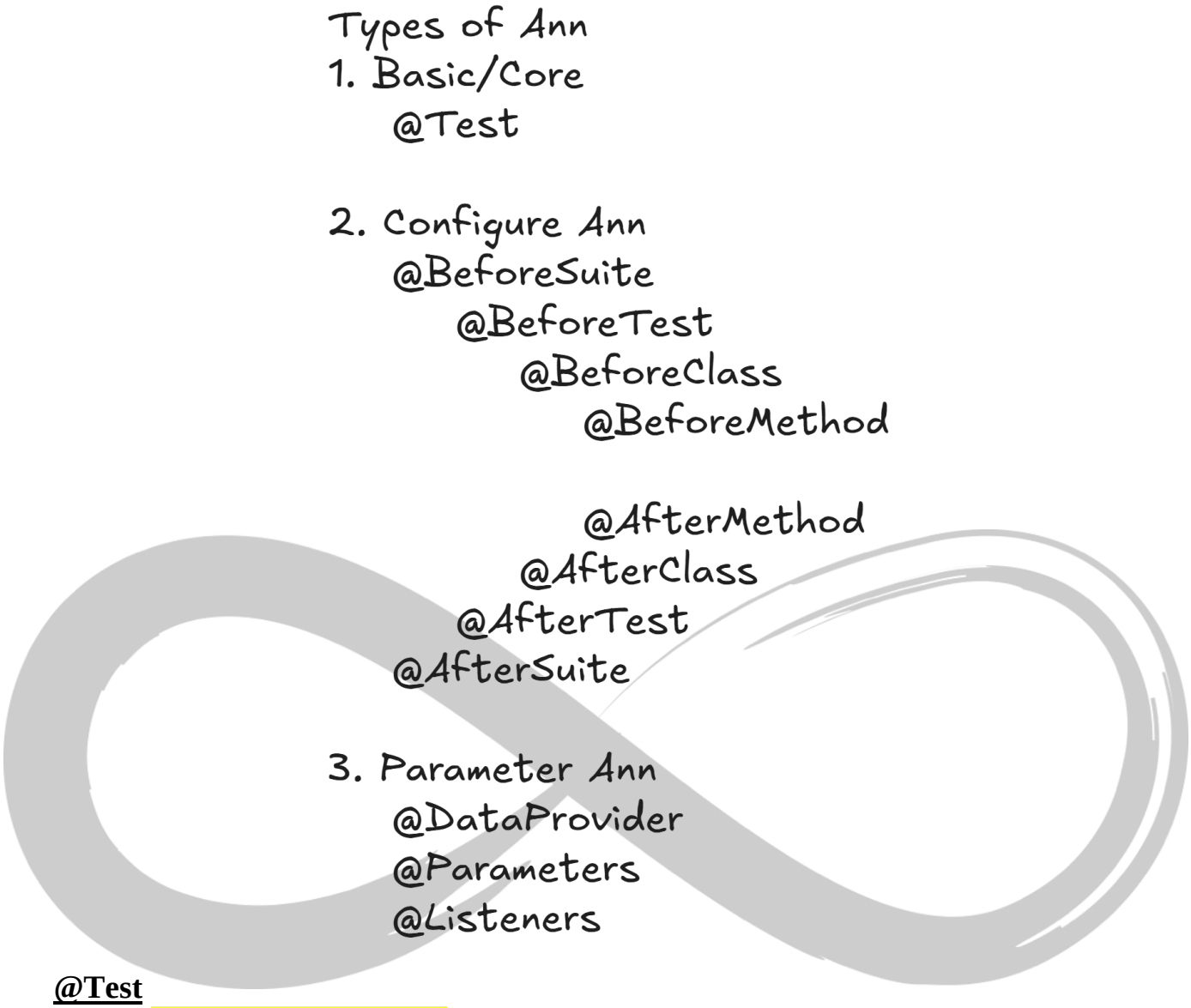
- Reporter.log("Step – 1", true);
- true will send the output in console also,
  by default it is false (Which doesn't send to console)

Before execution vs After execution





i. Annotations:-



@Test

1. It is a core/basic annotation of TestNG
2. Whenever we execute TestNG class, java Compiler/Interpreter always looks for @Test Annotation method to start the execution.
3. Without @Test method, TestNG class will not be executed.
4. @Test annotation method acts like a main method in TestNG class.
5. Annotation method return type should be "void",
   access specifier should be public,
   but method name can be anything.
6. As per the Rule of TestNG, class Name should end with "Test" suffix.
   (e.g., OrgTest, ContactTest…)
7. In one class we can keep multiple @Test annotation methods,
   but in real time we are going to maintain maximum 10 to 15 @test annotation methods, because maintenance will be easy.

ii. Assertion

Introduction

- It is a feature provided by TestNG.
- It is used to validate test scripts and used to verify expected results
- As per the rule of automation every expected result should be verified with statement, because java "if – else" statements doesn't have capability to fail the TestNG test script

```
If(true){
        Sop("first thing");
}else{
        Sop("another thing");
}
```

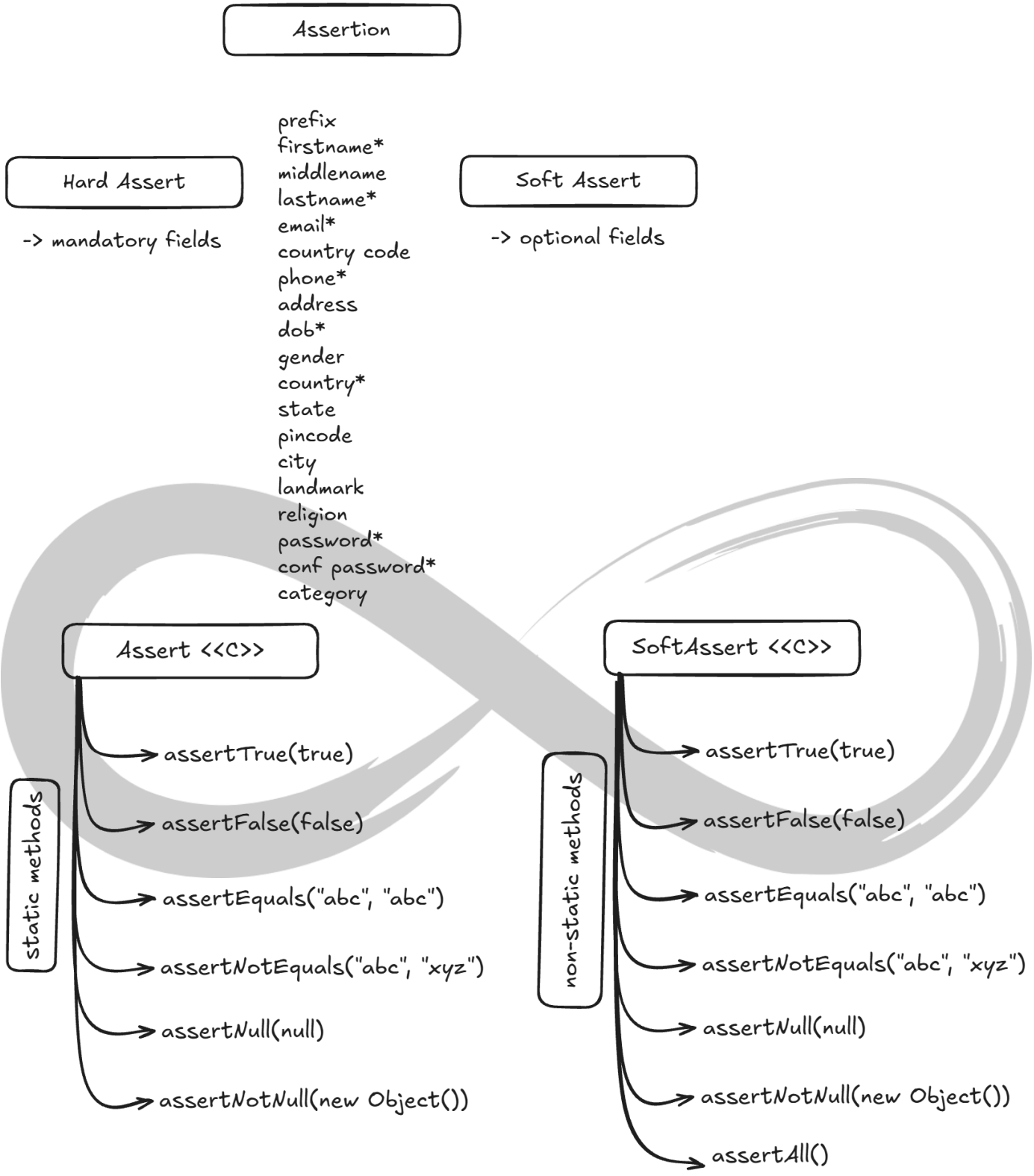
- Either one or another, There's nothing called Failure
- Java if else block can't say, Test case is getting failed
- So that, in order to verify the test case failed or not, we go for Assertion.

- There are 2 types of Assertions
    1. Hard Assert
    2. Soft Assert

- If any test case is getting failed, assertion will fail the TestNG test script by itself.

Difference between Hard and Soft Assert

Hard assert	Soft assert
Test execution will be aborted if the condition is not met	Test execution will be continued till the end even if the assert condition is not met.
Does not need to invoke any method to capture the assertion Exception -> java.lang.AssertionError	To view assertions result, at the end of the test case, we need to invoke assertAll(). Exception -> java.lang.AssertionError
All methods are static in nature	All methods are non-static in nature
Whole test case gets failed if at least 1 hard assert fails	assertAll extra line of code is required to track the failure status
to verify mandatory fields we go for hard assert -> email or username in Sign up	To verify non-mandatory fields we go for soft assert -> Middle name in Sign Up


Assertion
Hard Assert
-> mandatory fields
prefix
firstname*
middlename
lastname*
email*
country code
phone*
address
dob*
gender
country*
state
pincode
city
landmark
religion
password*
conf password*
category
Soft Assert
-> optional fields
Assert <<C>>
static methods
assertTrue(true)
assertFalse(false)
assertEquals("abc", "abc")
assertNotEquals("abc", "xyz")
assertNull(null)
assertNotNull(new Object())
SoftAssert <<C>>
non-static methods
assertTrue(true)
assertFalse(false)
assertEquals("abc", "abc")
assertNotEquals("abc", "xyz")
assertNull(null)
assertNotNull(new Object())
assertAll()

iii. Helper Attributes

1. Priority
2. DependsOnMethods
3. Enabled
4. AlwaysRun
5. InvocationCount
6. DataProvider
7. ThreadPoolSize

Priority

1. To change the "Order of Execution" we go for priority.
2. We can give positive, negative and 0.
3. lesser the value, earlier the execution.
4. If Priority is not specified, then the default value is 0.
5. Disadvantage:
    a. Illogical / senseless flow

dependsOnMethods

1. it is used to create dependency between 2 test scripts
   "test2" depends on "test1"
2. If "Test1" test script got passed,
   "Test2" execution will be continued.
3. If "Test1" test script got failed,
   "Test2" execution will be skipped.
4. The scope of dependsOnMethods is within particular TestNG Class.

Eg:-

```
@Test
public void test1() {
    Reporter.log("Account created Successfully");
}

@Test(dependsOnMethods= "test1")
public void test2(){
    Reporter.log("Account deleted Successfully");
}
```

5. Syntax:
   @Test(dependsOnMethods = "method Name")
6. Disadvantages:
   As per the rule of automation we should not create dependency between test cases.

invocationCount

1. Executing the same test script multiple times, with the same set of data

2. Default value is 1.
3. Disadvantage:
    a. We are executing the test scripts multiple time but with same data, what if we want different sets of data.

ThreadPoolSize

1. It is used to execute same test case multiple times with same test data **simultaneously**.
   In order to use this, invocation count is mandatory.
2. By default, threadPoolSize is 0.
   Eg:- @Test(invocationCount=10, threadPoolSize= 5)

Enabled

1. It is used to skip the test cases.
2. We can give Boolean values(True/False).
3. Default value is true.
4. If you want to skip some test cases, then you need to explicitly specify “false”.
   Eg:- @Test(enabled= false)

DataProvider

1. Executing the same test script multiple times, with the different set of test data
2. It always returns 2-D object array, because we can pass any type of datatype
3. It helps us to execute same test multiple time with the different set of data, each test script should have dedicated @DataProvider annotation
4. It plays major role in “data driven framework”, where we need to test the application with huge amount of data like ecommerce and booking application

iv. Types of Execution:

Batch Execution

1. Collection of multiple test scripts together is called as a batch, To execute multiple test scripts through the xml file on a single click without any manual intervention is called batch execution.
2. In order to achieve batch execution, we go for TestNG.xml configuration file
3. TestNG xml file always start with suite tag
4. In one xml file we can invoke N-number of TestNG classes, but all the classes should be present within a project.

How to create TestNG xml file automatically through eclipse?

1. Select all the TestNG classes or packages -> right click-> select TestNG and click on "Convert to TestNG" -> click on finish.
2. Automatically you will get the wizard to create xml file
3. Rename the file with your chosen name
4. Click on finish

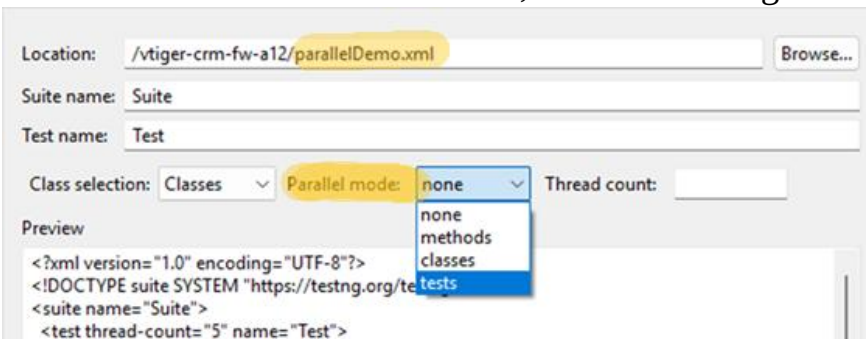
Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)
Suite					
SmokeTest	3	0	0	0	19,564

Class	Method	Start	Time (ms)
Suite			
SmokeTest — passed			
types_of_execution.Demo1	case1	1760168178027	6950
types_of_execution.Demo2	case2	1760168184986	6292
types_of_execution.Demo3	case3	1760168191282	6261

Parallel Execution:

Distributed Parallel execution:

- Distribute the test case across the multiple test runner & execute each <test>/Test runner in parallel is called Distributed Parallel execution
- we reduce the suite execution time, so that we can get the result early



- in order to achieve parallel execution we should enable parallel=”tests” , then create multiple test runner & distribute the test-cases

```
<suite parallel="tests" name="Suite">
    <test parallel="tests" name="TestThread1">
        <classes>
            <class name="types_of_execution.Demo1" >
                <methods>
                    <include name="case1"></include>
                </methods>
            </class>
        </classes>
    </test>

    <test parallel="tests" name="TestThread2">
        <classes>
            <class name="types_of_execution.Demo2" />
        </classes>
    </test>

    <test parallel="tests" name="TestThread3">
        <classes>
            <class name="types_of_execution.Demo3" />
        </classes>
    </test>
</suite>
```

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	In
Suite						
TestThread2	1	0	0	0	8,022	
TestThread1	1	0	0	0	8,062	
TestThread3	1	0	0	0	8,045	
Total	**3**	**0**	**0**	**0**	**24,129**	

Class	Method	Start	Time (ms)
Suite			
TestThread2 — passed			
types_of_execution.Demo2	case2	1760168723361	7965
TestThread1 — passed			
types_of_execution.Demo1	case1	1760168723362	8010
TestThread3 — passed			
types_of_execution.Demo3	case3	1760168723362	7993

-
- Thread count should be same as number of <test> testRunner

<parameter > is used to specify the browser data for each <test>

EG :

```
<suite name="Suite" parallel= "tests" thread-count="2">
        <test name="Test-Runner-Chrome">
                <parameter name="BROWSER" value="chrome"/>
                <classes>
```

```
            <class
            name="com.vtiger.comcast.organizationtest.CreateOrganizatio
            n"/>
            <class
            name="com.vtiger.comcast.organizationtest.SearchOrgTest"/>
        </classes>
    </test> <!-- Test -->
    <test name="Test-Runner-Firefox">
        <parameter name="BROWSER" value="firefox"/>
        <classes>
            <class name="com.vtiger.c
            omcast.organizationtest.CreateOrganization"/>
            <class
            name="com.vtiger.comcast.organizationtest.SearchOrgTest"/>
        </classes>
    </test>
</suite>
```

@Parameters annotation will be used to receive the data (Eg : browser , url, un, pwd etc.) from the XML file to test Scripts

Parallel Execution

Distributed parallel execution

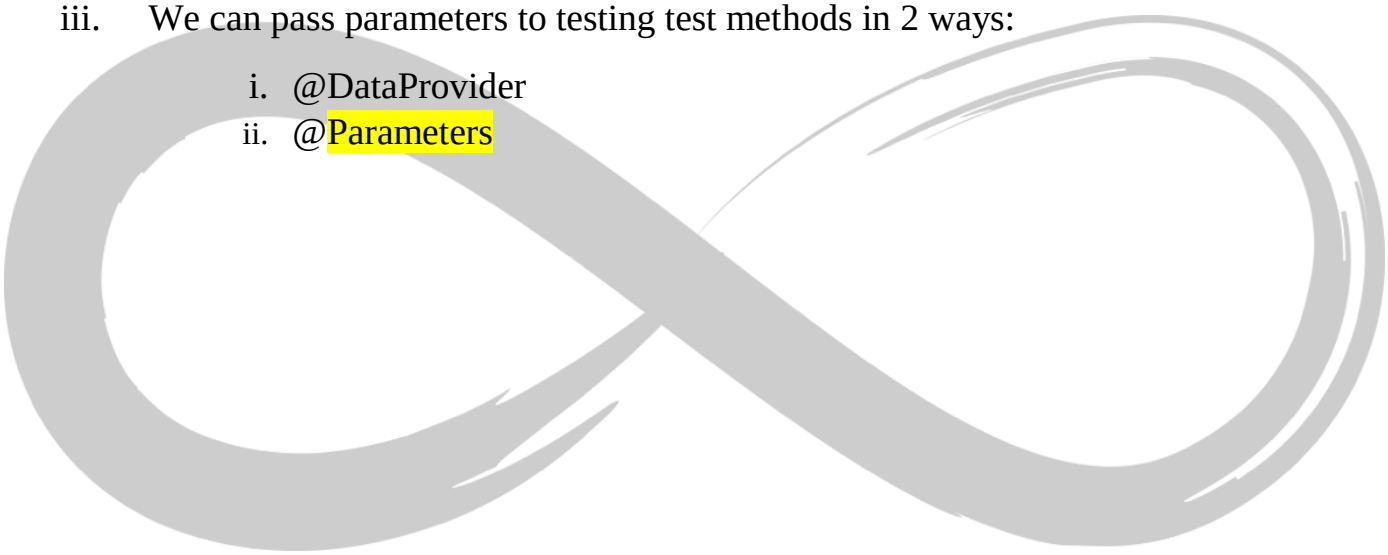
a. Distribute the test cases across the multiple test runner and execute each <test> test runner in parallel is called as Distributed parallel execution
b. We reduce the suite execution time, so that we can get the result early
c. In order to achieve parallel execution, we should enable parallel = “test” in <suite>, Then create multiple test runners and distribute the test cases

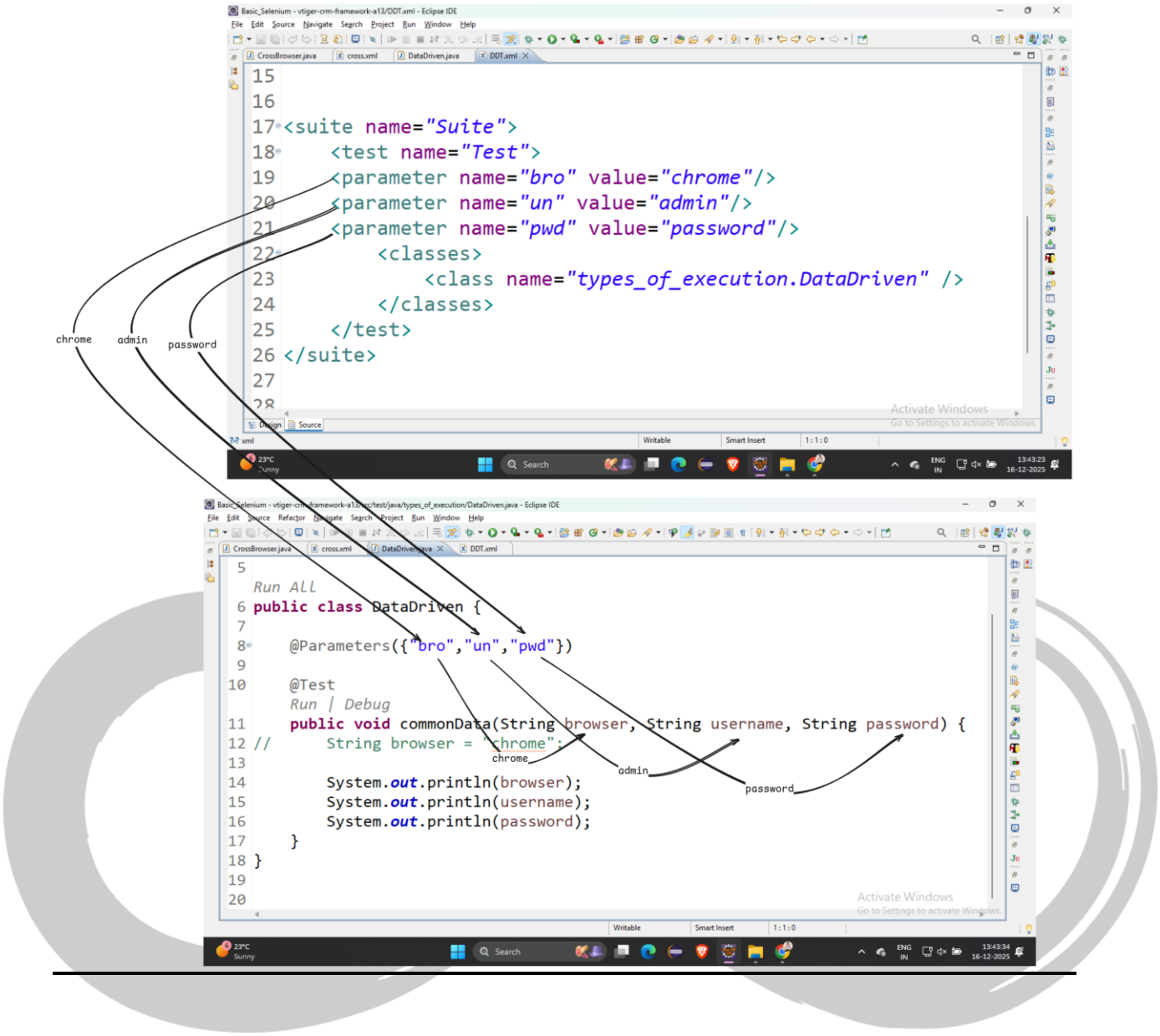
```
<suite parallel="tests" name="Suite">
  <test  parallel="tests" name="Test1">
      <classes>
          <class name="package.Demo1" />
      </classes>
  </test>
```

```
    <test parallel="tests" name="Test2">
        <classes>
            <class name="package.Demo2" />
        </classes>
    </test>
</suite>
```

Cross browser parallel execution / Browser Compatibility testing

i. Execute same set of testcase in different Browser parallel is called **cross browser testing**
ii. To achieve cross browser testing we should have <parameter> in XML file & @parameters annotation inside the testScript
iii. We can pass parameters to testing test methods in 2 ways:
    i. @DataProvider
    ii. @Parameters





Group Execution:

- Collection of similar test scripts across the TestNG classes is called grouping Execution
- In order to achieve grouping execution, each & every test script should have group name, group name will be written along with annotation
- In grouping execution, all configure annotation should have group name, otherwise those annotation will not participate in grouping execution like @BeforeSuite @BeforeClass, @BeforeMethode etc.

EG :

```
@BeforeSuite(groups = {"smokeTest","regressionTest"})
public void configBS() {
        System.out.println("======Execute BeforeSuite==========");
```

```
}
```

Smoke Test:

```
@Test(groups={"smokeTest"})
public void createCustomerTest(){
        System.out.println("execute createCustomerTest"); }

@Test(groups={"regressionTest")
public void modifyCustomerTest(){
        System.out.println("execute modifyCustomerTest"); }
```

In order to invoke grouping execution, it should declare Group Key in testng.xml file & group keep should be declared before <test>, after <suite> tag

Smoke Test:

```
<suite name="Suite">
        <groups>
                <run>
                        <include name="smokeTest"/>
                </run>
        </groups>
        <test name="Test">
                <classes>
                        <class name="pac1.ProjectAndCustomerTest"/>
                        <class name="pac2.ReportTest"/>
                </classes>
        <test>
</suite>
```

We can invoke multiple group-key in one XML File

```
<suite name="Suite">
        <groups>
                <run>
                        <include name="smokeTest"/>
                        <include name="regressionTest"/> </run>
        </groups>
        <test name="Test">
```

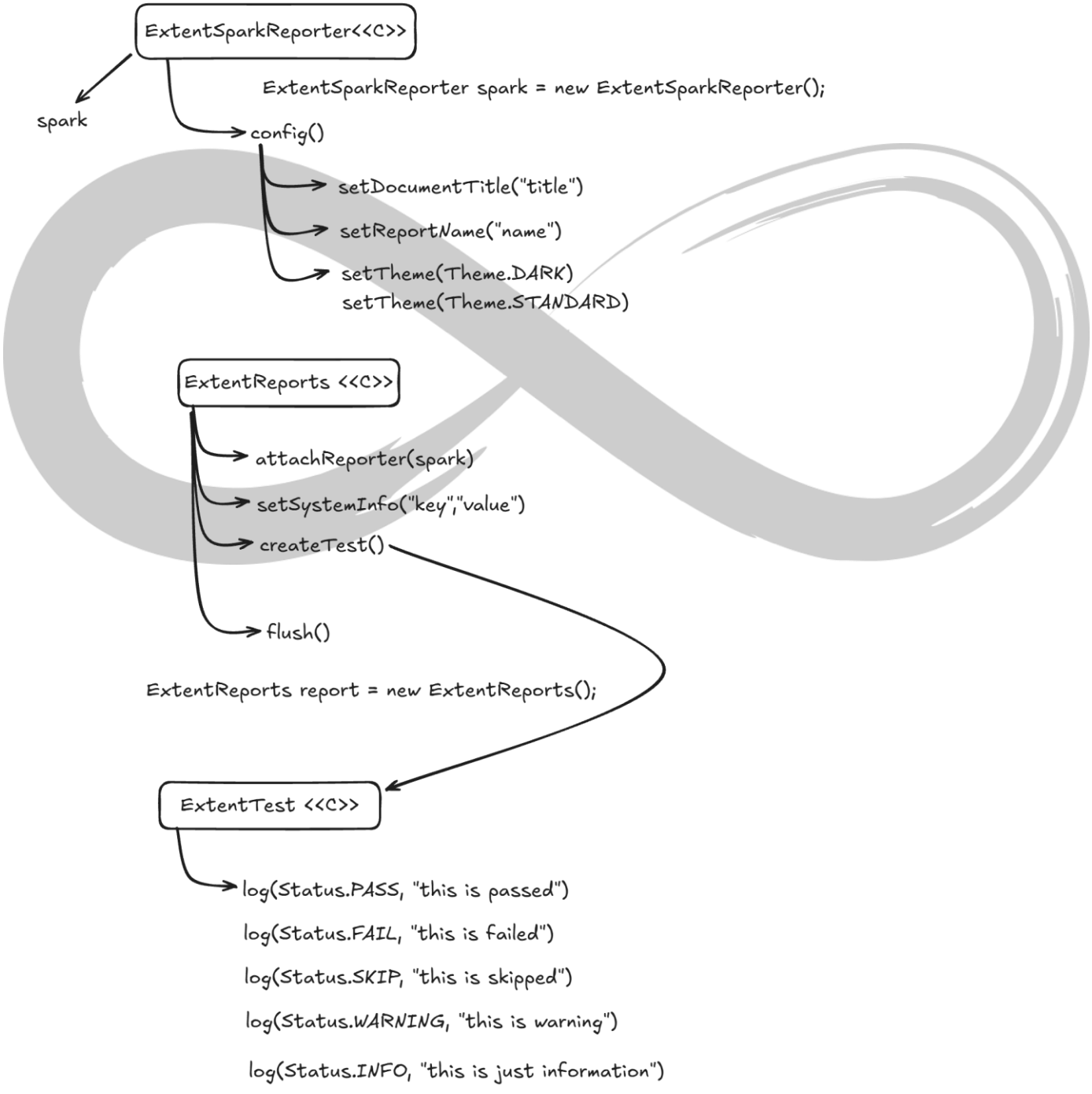
```
        <classes>
            <class name="pac1.ProjectAndCustomerTest"/>
            <class name="pac2.ReportTest"/>
        </classes>
</test>
</suite>
```

v. Advance Reports:

Extent Reports

- It provides advance features which helps in analyzing the test execution effectively

Advantages:
- It provides both high and low level reports
- It is an open source
- It provides eye-catching and lucid reports
- It has good documentation
- It can be easily integrated with CI/CD tools (jenkins)



vi. Configuration Annotations

What is annotation?

1. Annotation is the java template which is used to give information or metadata to compiler, developer and runtime environment.
2. Annotation can be used on top of variable, methods or classes (if allowed).
3. Annotation always start with @symbol

Types of Annotations

A. Basic annotation
    1. @Test
B. Configure annotation
    1. @BeforeMethod
    2. @AfterMethod
    3. @BeforeClass
    4. @AfterClass
    5. @BeforeTest
    6. @AfterTest
    7. @BeforeSuite
    8. @AfterSuite
C. Parameter annotation
    1. @Parameters
    2. @DataProvider
    3. @Listeners

Chronological Order of TestNG annotations

@BeforeSuite
    @BeforeTest
        @BeforeClass
            @BeforeMethod
                @Test (test case)
            @AfterMethod
        @AfterClass
    @AfterTest
@AfterSuite

NOTE: As per the Rule of the Automation, there should not be any dependency between two test cases, every test case should be unique (it means every test should have fresh login & logout code).

@BeforeMethod and @AfterMethod

1. Before method annotations will be executed, before executing each @Test method in a class
2. After method annotation will be executed, after executing each @Test in a class
3. BeforeMethod & AfterMethod will not be executed, without @test annotation method, because they are configuration method
4. In order to implement similar pre- condition for all the test cases like "LOGIN code" we go for @ BeforeMethod
5. In order to implement similar post-condition for all the test cases like "LOGOUT code" we go for @ AfterMethod

@BeforeClass & @AfterClass

1. BeforeClass Annotation method will be executed, before Executing first @test in a class
2. AfterClass Annotation method will be executed, after executing all/last test-case within a class
3. BeforeClass & AfterClass annotations will be executed only once in an entire class execution.
4. It will be used to develop global configuration like launch browser, object initialization

vii. Listeners

Introduction

1. It is the feature available in TestNG, which helps to capture runtime events.
2. It is TestNG annotation, that literally 'listens' to the event in script
3. Listeners are applied as interface in the code.

Listeners interfaces

ISuiteListener

- Works at suite level
- It listens and runs before the start and after the end of suite execution
- Abstract methods:
  1. onStart()
     executes before the @BeforeSuite
  2. onFinish()
     executes after the @AfterSuite

ITestListener

- Most frequently used testing listener
- Abstract methods:
  1. onTestStart()
  2. onTestSuccess()
  3. onTestSkipped()
  4. onTestFailure()

IRetryAnalyzer

- It helps user to retry the test script whenever test script is getting failed.
- In order to use this feature we have to implement IRetryAnalyzer interface and override retry().
- In retry(), we should specify the upper limit, so that test script will get executed till specified limit when the test is getting Failed again and again